BANCO ANDINO

Especificación de requerimientos

Motor de comprensión del lenguaje natural

Identificador de solicitudes

Dato	Valor
Documento	Especificación de requerimientos del software (ERS)
Sistema	Motor NLU — identificador de solicitudes
Asignatura	IA en Negocios Digitales
Entrega	Hito 1
Versión	1.0
Fecha	12 de noviembre de 2026
Estado	Consolidación del trabajo realizado hasta esta fecha

Este documento reúne el requerimiento del identificador de solicitudes de Banco Andino: qué debe hacer el motor, qué recibe el cliente, cómo se recorre un caso, qué decisión sale de cada mensaje y con qué modelo, contratos y arquitectura se sostiene esa decisión.

Ejemplo que recorre toda la especificación. «Llevo varios días esperando la devolución de mi dinero». Con solo ese texto, el motor reconoce una devolución pendiente, resume la espera y pide número de caso, monto y fecha. Si el mismo relato trae el caso 45821, $120.000 y la compra del 3 de noviembre, el caso se remite. Si además admite un cobro de tarjeta no reconocido, el caso pasa a un operador.

Contenido

1. Propósito y alcance
2. Actores y fronteras del sistema
3. Funcionalidades, beneficios y recorrido del cliente
4. Casos de uso
5. Requisitos funcionales y reglas de negocio
6. Contratos de la respuesta
7. Modelo de datos
8. Interacción de identidades
9. Secuencia de la identificación
10. Arquitectura de despliegue
11. Correspondencia con los artefactos del taller

1. Propósito y alcance

1.1 Propósito

El motor NLU interpreta un mensaje escrito por el cliente, reconoce la intención principal, extrae la información relevante, resume el problema y señala ambigüedades o datos insuficientes. Con esa lectura elige una sola acción: remitir el caso al siguiente proceso, pedir una aclaración o derivarlo a un operador.

La especificación fija ese comportamiento para el MVP. También deja escritos los contratos que la implementación debe producir, el modelo que conserva la solicitud mientras el cliente aclara, y la forma de desplegar el mismo flujo en AWS. El ejercicio de diagramas sobre una aplicación de transporte pertenece a la práctica de la Actividad 1 y queda fuera de este sistema.

1.2 Qué termina el identificador

El identificador termina cuando el mensaje ya tiene una decisión. La respuesta de esa decisión incluye la intención, la confianza, los datos encontrados, los datos que faltan, un resumen breve, la acción elegida y el motivo. La atención posterior del proceso de negocio y la conversación del operador con el cliente empiezan después de ese punto.

1.3 Qué queda dentro del diseño de soporte

El canal, la persistencia y el alojamiento acompañan esa decisión y forman parte de lo ya definido en el taller. La interfaz captura el texto y muestra el resultado. La API correlaciona la petición y proyecta el estado. Cada aclaración del cliente vuelve a entrar por el mismo servicio y produce una interpretación nueva sobre la misma solicitud.

1.4 Fuera del alcance de esta entrega

- El inicio de sesión. El diagrama distingue al visitante del usuario logueado, y el alta de la sesión queda fuera del identificador.
- La resolución del trámite una vez remitido: abonos, investigación del caso o tiempos de respuesta del área de negocio.
- La atención que el operador da después de tomar el caso. El motor le entrega el resumen y el motivo; ahí termina su trabajo.
- Umbrales numéricos de confianza. El taller fija que la confianza acompaña a la intención y que una lectura insegura deriva el caso. No fija un valor de corte.
- El catálogo completo de intenciones del banco. El caso elaborado es la devolución pendiente, y el diseño admite otras lecturas cuando el texto las pone en competencia.

2. Actores y fronteras del sistema

Cliente es un actor abstracto. Usuario visitante y usuario logueado lo especializan. Los dos heredan el envío del mensaje y la obtención de una decisión. Solo el usuario logueado se asocia, además, a remitir el caso. Proceso de negocio y operador son actores secundarios: reciben la decisión y su trabajo posterior queda fuera.

Actor	Papel en el identificador
Cliente	Actor abstracto. El visitante y el logueado heredan su asociación con el envío del mensaje.
Usuario visitante	El mensaje llega sin identidad de cliente. Esa identidad cuenta como dato faltante, así que la salida de remisión no está disponible para él.

Actor	Papel en el identificador
Usuario logueado	La sesión ya aporta la identidad. Puede obtener la remisión cuando la intención es única, la confianza alcanza y están el caso, el monto y la fecha.
Proceso de negocio	Recibe el caso cuando la acción es remitir. Continúa el trámite fuera del motor.
Operador	Recibe el caso cuando la acción es derivar, con el resumen y el motivo ya armados.

2.1 Cadena de entrada

La cadena de entrada es lineal. El cliente solo conoce la interfaz, la interfaz solo conoce la API y la API solo conoce al coordinador. A partir de ahí el coordinador es la única identidad que alcanza a los agentes de comprensión, al validador y a los dos destinos. React envía el texto a POST /api/solicitudes. La API se lo entrega al coordinador. El coordinador dispara el resto del flujo y devuelve un único resultado.

Identidad	Capa	Trabajo que conserva	Habla solo con
Cliente	Canal	Redactar la solicitud y leer la respuesta	Interfaz web
Interfaz web (React)	Canal	Mostrar resumen, preguntas o aviso de derivación	Cliente y API
API de solicitudes (Node)	Borde	Correlacionar la petición y el estado HTTP	Interfaz y coordinador
Agente coordinador	Orquestación	Integrar las salidas y enrutar según el veredicto	API, cuatro agentes y dos destinos
Agente intérprete	Comprensión	Clasificar la intención y estimar la confianza	Coordinador
Agente extractor	Comprensión	Obtener entidades, identificadores y ausencias	Coordinador
Agente de análisis contextual	Comprensión	Resumir, detectar ambigüedades y juzgar suficiencia	Coordinador
Agente validador	Control	Contrastar la salida con las reglas y elegir la acción	Coordinador
Proceso de negocio	Destino	Continuar el caso ya válido	Coordinador
Operador humano	Destino	Atender lo que el NLU no cierra con el cliente	Coordinador

3. Funcionalidades, beneficios y recorrido del cliente

3.1 Funcionalidades, de mayor a menor prioridad

La prioridad la marca lo que permite actuar sobre la solicitud. Primero hay que entender qué pide el cliente y con qué datos. Después, si esa lectura es segura y completa. Al final, qué hacer con ella y cómo dejar constancia.

Orden	Funcionalidad	Para qué existe
1	Clasificar la intención principal	Reconocer de qué se trata el mensaje y devolver un tipo de solicitud. Sin esa etiqueta el flujo no tiene destino.

Orden	Funcionalidad	Para qué existe
2	Extraer la información relevante	Obtener entidades, identificadores y atributos, y señalar los que no aparecen. La intención sola no abre un trámite.
3	Estimar la confianza	Acompañar la intención con un grado de certeza. Una lectura poco segura no pasa al siguiente proceso.
4	Juzgar suficiencia y ambigüedad	Decidir si los datos bastan para un caso concreto y marcar lecturas dudosas o contradictorias.
5	Resumir el problema	Redactar en pocas líneas qué ocurre, para que el destino trabaje sobre la lectura y no sobre el mensaje crudo.
6	Decidir el destino	Elegir una sola acción —remitir, pedir aclaración o derivar— y dejar el motivo.
7	Formular la aclaración	Cuando faltan datos que el cliente puede aportar, devolver preguntas concretas.
8	Conservar el contexto	Tratar la respuesta del cliente como continuación de la misma solicitud y volver a interpretar, extraer y validar sobre el mensaje enriquecido.
9	Dejar trazabilidad	Registrar intención, datos, resumen, confianza y motivo. Sirve para auditar y corregir el motor.

3.2 Beneficios para el cliente

El cliente cuenta el problema con sus palabras y recibe una respuesta útil en el mismo momento: el trámite sigue, le piden solo lo que falta o una persona toma el caso ya entendido.

- Explica el caso como lo vive. Escribe la espera de una devolución y el banco reconoce esa intención. No necesita saber el nombre interno del trámite ni recorrer un menú.
- Llega al proceso correcto en el primer mensaje cuando la solicitud está clara y trae los datos necesarios. Evita repetir la historia en otro canal.
- Aporta menos datos de los que ya dijo. El motor toma del texto el producto, el monto, la fecha o el número de caso.
- Recibe preguntas concretas cuando algo falta: número de caso, monto o fecha. Puede responder en el mismo hilo.
- Cuando el mensaje admite más de una lectura o la certeza es baja, una persona recibe el caso con el resumen ya armado.
- La aclaración se suma al mensaje original. No abre un trámite nuevo ni pierde lo ya contado.
- La respuesta le dice si la solicitud fue remitida, si está incompleta o si pasó a un operador.

3.3 Recorrido del cliente

El recorrido normal es el de una solicitud cuyo primer mensaje todavía no cierra la identificación. El cliente cuenta el caso, ve qué entendió el banco, completa solo lo que falta y recibe el destino final en el mismo hilo.

1. Entra al canal. Abre la web del banco, inicia sesión y llega a la pantalla para escribir una solicitud. Ve un campo de texto libre y el botón para enviarla. El inicio de sesión en sí queda fuera del identificador.
2. Redacta con sus palabras. No elige un tipo de trámite ni llena un formulario previo.
3. Envía la solicitud. La interfaz manda el texto y el cliente espera en la misma pantalla.
4. Ve qué se entendió. La respuesta muestra el resumen y la lectura, para que compruebe si el banco interpretó el caso.

5. Recibe las preguntas que faltan. En el ejemplo, la pantalla pide número de caso, monto y fecha. No le pide repetir el relato.
6. Responde en el mismo hilo. El banco suma esos datos al mensaje original y vuelve a leer la solicitud completa.
7. Recibe el cierre de la identificación. Si el caso queda claro, la pantalla muestra el resumen actualizado e indica que la solicitud fue remitida al proceso que la resuelve.

En ese mismo punto de respuesta hay otras dos salidas posibles.

- Sigue incompleta. Si aún falta algo que el cliente puede aclarar, la pantalla hace otra pregunta concreta y el paso 6 se repite.
- Pasa a una persona. Si el mensaje admite más de una lectura o la certeza es baja, la pantalla avisa que un operador continúa la atención. El cliente no vuelve a contar el caso.

4. Casos de uso

El MVP se cubre con seis casos de uso de comprensión y tres extensiones de salida. Enviar el mensaje y obtener una decisión es la única entrada. Clasificar, extraer, estimar la confianza, resumir y elegir una acción ocurren siempre, como inclusiones. De las tres extensiones se ejecuta exactamente una, según la condición del caso: remitir, pedir una aclaración o derivar a un operador.

Caso de uso	Qué hace	Relación
UC1 Enviar el mensaje y obtener una decisión	El cliente envía un texto. El motor responde una sola vez con intención, confianza, datos encontrados, datos faltantes, resumen, acción y motivo.	Caso base. Lo heredan visitante y logueado.
UC2 Clasificar la intención	Asigna la etiqueta principal. Si el texto admite otra lectura igual de plausible, conserva esa duda para el veredicto.	<<include>> de UC1
UC3 Extraer datos encontrados y faltantes	Obtiene lo que aparece y marca lo que el caso exige y no está.	<<include>> de UC1
UC4 Estimar la confianza	Acompaña la intención con un grado de certeza. Un texto de una sola lectura la mantiene alta. Dos lecturas o un texto vago la bajan.	<<include>> de UC1
UC5 Resumir la solicitud	Redacta en una o dos frases lo entendido, para que el destino trabaje sobre la lectura.	<<include>> de UC1
UC6 Elegir una sola acción	Selecciona remitir, pedir aclaración o derivar, y deja el motivo.	<<include>> de UC1
UC7 Remitir el caso	La intención es única, la confianza alcanza y están los datos mínimos. Solo el usuario logueado.	<<extend>> de UC6. Lo recibe el proceso de negocio.
UC8 Pedir una aclaración	La intención se sostiene y falta un dato que el cliente puede aportar. En el visitante, la identidad cuenta como dato faltante.	<<extend>> de UC6
UC9 Derivar a un operador	Hay más de una lectura, o la confianza es baja. Vale para los dos actores.	<<extend>> de UC6. Lo recibe el operador.

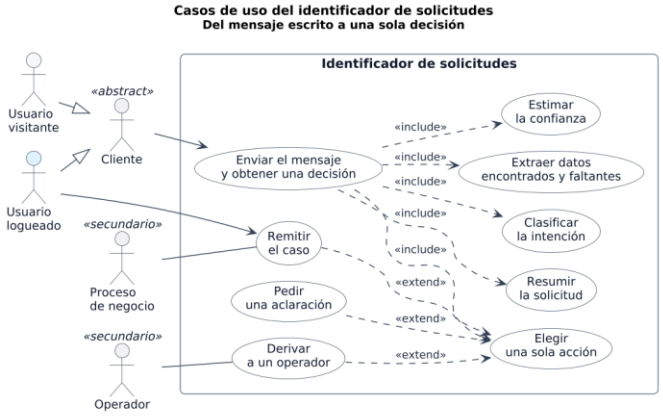
La poscondición de UC1 es una respuesta con intención, confianza, datos encontrados, datos faltantes, resumen, acción y motivo. El sistema termina en ese punto.

4.1 Las tres salidas con el mismo relato

Salida	Mensaje	Lectura y motivo
Remitir	«Llevo varios días esperando la devolución de mi dinero del caso 45821, por $120.000, de la compra del 3 de noviembre.»	Intención única de devolución pendiente, confianza alta, datos mínimos completos. El usuario está logueado.
Pedir aclaración	«Llevo varios días esperando la devolución de mi dinero.»	La intención se sostiene. Faltan caso, monto y fecha. La respuesta nombra esas tres preguntas.
Derivar	«Llevo varios días esperando la devolución de mi dinero, o quizá fue un cobro de la tarjeta que no reconozco.»	Compiten una devolución pendiente y una transacción no reconocida. La confianza no alcanza para elegir una.

4.2 Diagrama de casos de uso

El diagrama separa al visitante del usuario logueado, marca Cliente como actor abstracto y deja a proceso de negocio y operador como actores secundarios. La gráfica proviene de la fuente PlantUML que sigue a la figura.



Elemento	Definición
Cliente	Actor abstracto. El visitante y el logueado heredan su asociación.
Usuario visitante	El mensaje llega sin identidad de cliente.
Usuario logueado	La sesión ya aporta la identidad. Iniciar sesión queda fuera del sistema.
«include»	Clasificar, extraer, estimar la confianza, resumir y elegir acción ocurren siempre.
«extend»	Se ejecuta exactamente una: remitir, pedir aclaración o derivar.
Remitir el caso	Solo usuario logueado, con una intención, confianza suficiente y datos mínimos (caso, monto y fecha).
Pedir una aclaración	Una intención, confianza suficiente y un dato que el cliente puede aportar. En el visitante, la identidad cuenta como dato faltante.
Derivar a un operador	Más de una lectura, o confianza baja. Vale para los dos actores.
Actores secundarios	Proceso de negocio y Operador reciben la decisión. Su trabajo posterior queda fuera.
Poscondición	Una respuesta: intención, confianza, datos encontrados, datos faltantes, resumen, acción y motivo.
Ejemplo	«Llevo varios días esperando la devolución de mi dinero». Solo ese texto: pedir aclaración. Logueado, con caso 45821, $120.000 y 3 de noviembre: remitir. El mismo relato más un cobro no reconocido: derivar.

Figura 1. Casos de uso del identificador de solicitudes, del mensaje escrito a una sola decisión.

Fuente PlantUML — casos de uso

```
' Casos de uso — Identificador de solicitudes (MVP)
' El sistema termina cuando el mensaje tiene una sola decisión.
'
' Cliente es abstracto: no se instancia.
' Usuario visitante y Usuario logueado lo especializan y heredan
' «Enviar el mensaje y obtener una decisión».
' Solo el usuario logueado se asocia además a «Remitir el caso».
'
' <<include>>  comportamiento siempre presente en el caso base.
' <<extend>>   exactamente una extensión, según la condición del cuadro.
' Proceso de negocio y Operador reciben la decisión; no actúan después
' dentro de este sistema.

@startuml casos_de_uso
!pragma layout smetana

title Casos de uso del identificador de solicitudes\n<size:13>Del mensaje escrito a una sola decisión</size>

skinparam shadowing false
skinparam roundcorner 10
```

```
skinparam linetype ortho
skinparam nodesep 18
skinparam ranksep 22
skinparam dpi 140
skinparam ArrowColor #334155
skinparam ArrowFontColor #1E293B
skinparam ArrowFontSize 12

skinparam actor {
  BorderColor #334155
  FontColor #0F172A
  FontSize 13
}
skinparam usecase {
  BackgroundColor #F8FAFC
  BorderColor #334155
  FontColor #0F172A
  FontSize 13
}
skinparam rectangle {
  BackgroundColor #FFFFFF
  BorderColor #64748B
  FontColor #0F172A
  FontStyle bold
  FontSize 14
}
skinparam legend {
  BackgroundColor #F8FAFC
  BorderColor #64748B
  FontColor #0F172A
  FontSize 12
}

left to right direction

actor "Usuario\nvisitante" as visitante #F1F5F9
actor "Cliente" as cliente <<abstract>>
actor "Usuario\nlogueado" as logueado #E0F2FE

visitante --|> cliente
logueado --|> cliente

rectangle "Identificador de solicitudes" {
  usecase "Enviar el mensaje\ny obtener una decisión" as UC1

  together {
    usecase "Clasificar\nla intención" as UC2
    usecase "Extraer datos\nencontrados y faltantes" as UC3
    usecase "Estimar\nla confianza" as UC4
    usecase "Resumir\nla solicitud" as UC5
    usecase "Elegir\nuna sola acción" as UC6
  }

  together {
    usecase "Remitir\nel caso" as UC7
    usecase "Pedir\nuna aclaración" as UC8
    usecase "Derivar\na un operador" as UC9
  }

  UC1 ..> UC2 : <<include>>
  UC1 ..> UC3 : <<include>>
  UC1 ..> UC4 : <<include>>
  UC1 ..> UC5 : <<include>>
  UC1 ..> UC6 : <<include>>

  UC7 ..> UC6 : <<extend>>
  UC8 ..> UC6 : <<extend>>
  UC9 ..> UC6 : <<extend>>
}

cliente --> UC1
logueado --> UC7

actor "Proceso\nde negocio" as proceso <<secundario>>
```

```
actor "Operador" as operador <<secundario>>
proceso -- UC7
operador -- UC9

legend bottom
  |= Elemento |= Definición |
  | Cliente | Actor abstracto. El visitante y el logueado heredan su asociación. |
  | Usuario visitante | El mensaje llega sin identidad de cliente. |
  | Usuario logueado | La sesión ya aporta la identidad. Iniciar sesión queda fuera del sistema. |
  | <<include>> | Clasificar, extraer, estimar la confianza, resumir y elegir acción ocurren siempre. |
  | <<extend>> | Se ejecuta exactamente una: remitir, pedir aclaración o derivar. |
  | Remitir el caso | Solo usuario logueado, con una intención, confianza suficiente y datos mínimos (caso,
monto y fecha). |
  | Pedir una aclaración | Una intención, confianza suficiente y un dato que el cliente puede aportar. En el
visitante, la identidad cuenta como dato faltante. |
  | Derivar a un operador | Más de una lectura, o confianza baja. Vale para los dos actores. |
  | Actores secundarios | Proceso de negocio y Operador reciben la decisión. Su trabajo posterior queda fuera. |
  | Poscondición | Una respuesta: intención, confianza, datos encontrados, datos faltantes, resumen, acción y
motivo. |
  | Ejemplo | «Llevo varios días esperando la devolución de mi dinero». Solo ese texto: pedir aclaración.
Logueado, con caso 45821, $120.000 y 3 de noviembre: remitir. El mismo relato más un cobro no reconocido:
derivar. |
endlegend

@enduml
```

5. Requisitos funcionales y reglas de negocio

5.1 Requisitos funcionales

ID	Requisito	Origen
RF-01	Recibir un mensaje en lenguaje natural y devolver una sola decisión.	UC1
RF-02	Clasificar la intención principal y conservar la duda cuando otra lectura es igual de plausible.	UC2
RF-03	Extraer entidades, identificadores y atributos presentes, y marcar los atributos ausentes.	UC3
RF-04	Estimar la confianza de la intención clasificada.	UC4
RF-05	Redactar un resumen de una o dos frases con lo entendido.	UC5
RF-06	Elegir exactamente una acción y registrar su motivo.	UC6
RF-07	Remitir el caso al proceso de negocio cuando el cliente está logueado, la intención es única, la confianza es suficiente y están los datos mínimos del caso.	UC7
RF-08	Pedir aclaración con preguntas concretas cuando la intención se sostiene y falta un dato que el cliente puede aportar.	UC8
RF-09	Derivar a un operador cuando hay más de una lectura o la confianza es baja, para visitante y logueado.	UC9
RF-10	Tratar la aclaración como continuación de la misma solicitud: el mensaje enriquecido vuelve a entrar por POST /api/solicitudes.	Funcionalidad 8
RF-11	Registrar intención, datos, resumen, confianza y motivo del veredicto para auditoría.	Funcionalidad 9
RF-12	Mostrar al cliente el resumen y el estado resultante: remitida, incompleta o derivada.	Recorrido del cliente
RF-13	Correlacionar cada POST /api/solicitudes con la solicitud y responder HTTP 200 en las tres salidas. El destino se distingue en el cuerpo, no en el código de estado.	Secuencia

5.2 Reglas de negocio y de diseño

ID	Regla
RN-01	De cada mensaje sale una sola acción: REMITIR_PROCESO, SOLICITAR_INFORMACION o DERIVAR_OPERADOR.
RN-02	Para una devolución pendiente, los datos mínimos de remisión son número de caso, monto y fecha. El producto se extrae cuando aparece en el texto.
RN-03	El usuario visitante no remite el caso. La ausencia de identidad cuenta como dato faltante y la salida correspondiente es pedir aclaración, salvo que el mensaje deba derivarse.
RN-04	Intérprete y extractor trabajan a la vez sobre el texto crudo. El extractor no espera la intención, para poder detectar después un desajuste entre lo clasificado y lo extraído.
RN-05	El análisis contextual ocurre después de las dos salidas anteriores. Resume, marca ambigüedades y juzga si la información alcanza.
RN-06	El coordinador integra la salida. El validador decide la acción. Clasificar, extraer y resumir no autorizan por sí solos el paso al siguiente proceso.
RN-07	SOLICITAR_INFORMACION no abre otro flujo ni otra identidad. La aclaración reutiliza la misma cadena de entrada.
RN-08	REMITIR_PROCESO publica hacia el proceso de negocio. DERIVAR_OPERADOR deja el caso para el operador. Esas dos publicaciones salen del coordinador, que ya recibió el veredicto.
RN-09	Los agentes de comprensión no se invocan entre sí. El validador no habla con el proceso de negocio ni con el operador. La interfaz no llama a los agentes.
RN-10	Cada aclaración genera una interpretación nueva. La interpretación anterior se conserva; no se reescribe.
RN-11	El estado visible de la solicitud es la proyección de la última decisión.

5.3 Datos mínimos del ejemplo

Sobre «Llevo varios días esperando la devolución de mi dinero», el análisis contextual deja esta lectura: intención de consulta por devolución pendiente, resumen de un dinero que no ha vuelto tras varios días, información insuficiente porque faltan caso, monto y fecha. La acción es pedir aclaración y el motivo es que la intención es clara y el cliente puede aportar esos tres datos.

6. Contratos de la respuesta

Cada flecha de retorno del diagrama de secuencia nombra el objeto que la implementación debe producir. Esos nombres en español son el contrato del flujo. El modelo de datos del capítulo 7 usa los nombres en inglés de las entidades persistidas. La tabla de equivalencias cierra los dos vocabularios.

Objeto	Campos
Intencion	tipo, confianza
DatosRelevantes	entidades, identificadores, atributosAusentes
AnalisisContextual	resumen, ambiguedades, datosFaltantes, suficiente
SalidaIntegrada	intencion, datos, analisis
Veredicto	cumpleRequisitos, accion, motivo

accion solo admite REMITIR_PROCESO, SOLICITAR_INFORMACION o DERIVAR_OPERADOR. Cuando la acción pide aclaración, la respuesta al cliente incluye además las preguntas.

6.1 Equivalencia de nombres

En el flujo y en la API	En el modelo persistido	Significado
RECIBIDA	RECEIVED	La solicitud existe y el mensaje acaba de entrar.
INCOMPLETA	INCOMPLETE	La última decisión pidió una aclaración.
REMITIDA	FORWARDED	La última decisión remitió el caso.
DERIVADA	ESCALATED	La última decisión dejó el caso al operador.
REMITIR_PROCESO	FORWARD_TO_PROCESS	Publicar el caso hacia el proceso de negocio.
SOLICITAR_INFORMACION	REQUEST_INFORMATION	Devolver preguntas al cliente por la misma llamada.
DERIVAR_OPERADOR	ESCALATE_TO_OPERATOR	Dejar el caso en la cola del operador.

6.2 Cuerpo de la respuesta HTTP

Las tres ramas responden 200. El cuerpo distingue el destino y transporta la decisión completa que exige el MVP.

Campo de la respuesta	Origen
estado	REMITIDA, INCOMPLETA o DERIVADA
intencion.tipo y confianza	Intencion
datos encontrados	entidades e identificadores
datos faltantes	atributosAusentes y datosFaltantes
resumen	AnalisisContextual.resumen
accion y motivo	Veredicto
preguntas	Solo cuando la acción es SOLICITAR_INFORMACION

7. Modelo de datos

Tres entidades sostienen el identificador. Request es la solicitud que el cliente va construyendo. Interpretation es lo que el motor entendió en un paso. Decision es la única acción que sale de esa interpretación. La solicitud es la raíz. Vive mientras el cliente aclara el caso y concentra el estado visible del trámite.

7.1 Request

El texto no vive aquí como un único string definitivo. Cada envío del cliente es una vuelta sobre la misma solicitud.

Campo	Rol
id	Identidad del caso.
status	RECEIVED, INCOMPLETE, FORWARDED o ESCALATED.
correlationId	Une el POST /api/solicitudes con la respuesta.
createdAt	Orden y auditoría del hilo.

7.2 Interpretation

Es el resultado de un paso de comprensión sobre el mensaje ya enriquecido. No decide el destino. Intención, entidades y resumen son valores de esta interpretación: si el cliente aclara, se produce otra interpretación y la anterior permanece.

Campo	Rol
intentType	Etiqueta principal, por ejemplo devolución pendiente.
confidence	Certeza de esa etiqueta.
entities	Lo que sí aparece: espera, dinero, devolución.
identifiers	Caso, monto, fecha o producto cuando vienen en el texto.
missingFields	Lo que el caso exige y no está.
summary	Una o dos frases de lo entendido.
ambiguities	Lecturas en pugna.
sufficient	Si los datos alcanzan para un caso concreto.

7.3 Decision

Es el veredicto de una interpretación. El motor responde una sola vez con ella.

Campo	Rol
action	FORWARD_TO_PROCESS, REQUEST_INFORMATION o ESCALATE_TO_OPERATOR.
meetsRequirements	Si la interpretación pasa las reglas del caso.
reason	Por qué se eligió esa acción.
questions	Solo cuando la acción pide aclaración. En el ejemplo: caso, monto y fecha.

7.4 Relaciones

- Una Request acumula cero o varias Interpretation. El primer mensaje y cada aclaración vuelven a entrar por el mismo POST y generan una interpretación nueva.
- Cada Interpretation produce una sola Decision.
- El status de Request es la proyección de la última decisión: remitir deja FORWARDED, pedir datos deja INCOMPLETE y derivar deja ESCALATED.

Con el ejemplo, la solicitud nace en RECEIVED. La interpretación marca intención de devolución pendiente, confianza suficiente para esa etiqueta, faltantes de caso, monto y fecha, y sufficient en falso. La decisión pide aclaración y nombra esas tres preguntas. Si el cliente responde en el mismo hilo, nace otra interpretación sobre el mismo caso.

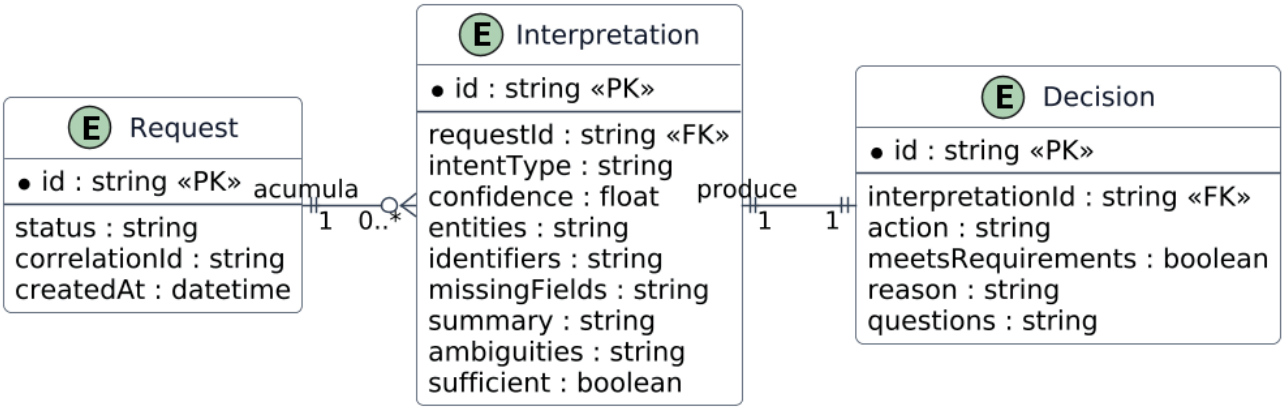


Figura 2. Request acumula interpretaciones y cada interpretación produce una decisión. La gráfica está en horizontal para la lectura impresa; la fuente del modelo es el Mermaid siguiente.

Fuente Mermaid — modelo de datos

```
erDiagram
    Request ||--o{ Interpretation : "acumula"
    Interpretation ||--|| Decision : "produce"

    Request {
        string id PK
        string status "RECEIVED | INCOMPLETE | FORWARDED | ESCALATED"
        string correlationId
        datetime createdAt
    }

    Interpretation {
        string id PK
        string requestId FK
        string intentType
        float confidence
        string entities
        string identifiers
        string missingFields
        string summary
        string ambiguities
        boolean sufficient
    }

    Decision {
        string id PK
        string interpretationId FK
        string action "FORWARD_TO_PROCESS | REQUEST_INFORMATION | ESCALATE_TO_OPERATOR"
        boolean meetsRequirements
        string reason
        string questions
    }
```

8. Interacción de identidades

Este diagrama fija quién puede hablar con quién. El de secuencia, en el capítulo siguiente, ordena los mismos mensajes en el tiempo. Cada caja es una identidad, escrita como instancia : tipo. Cada flecha es una interacción permitida. Si dos identidades no están unidas, no se hablan: el intérprete no llama al extractor, el análisis no llama al validador y el validador no remite el caso ni lo deriva.

La numeración de las flechas es la del diagrama de secuencia, para cruzar ambos sin cambiar de contrato. Los números que no aparecen en la tabla son trabajo interno de una identidad o la respuesta de una flecha que ya está listada.

Flecha	De	Hacia	Mensaje	Vuelta

Flecha	De	Hacia	Mensaje	Vuelta
1	Cliente	Interfaz	solicitud	resultado
2	Interfaz	API	POST /api/solicitudes	200
4	API	Coordinador	analizar(mensaje)	Resultado
5	Coordinador	Intérprete	identificarIntencion	Intencion
8	Coordinador	Extractor	extraerDatos	DatosRelevantes
11	Coordinador	Análisis	analizarContexto	AnalisisContextual
15	Coordinador	Validador	validar	Veredicto
18	Coordinador	Proceso	remitir	Acuse, solo si la acción es REMITIR_PROCESO
26	Coordinador	Operador	derivar	Acuse, solo si la acción es DERIVAR_OPERADOR

Intérprete y extractor solo se conectan al coordinador, así que pueden ejecutarse a la vez sobre el mismo mensaje. La flecha 15 devuelve el veredicto. Las flechas 18 y 26 salen del coordinador. Si la acción es SOLICITAR_INFORMACION, la vuelta al cliente usa las flechas 4, 2 y 1.

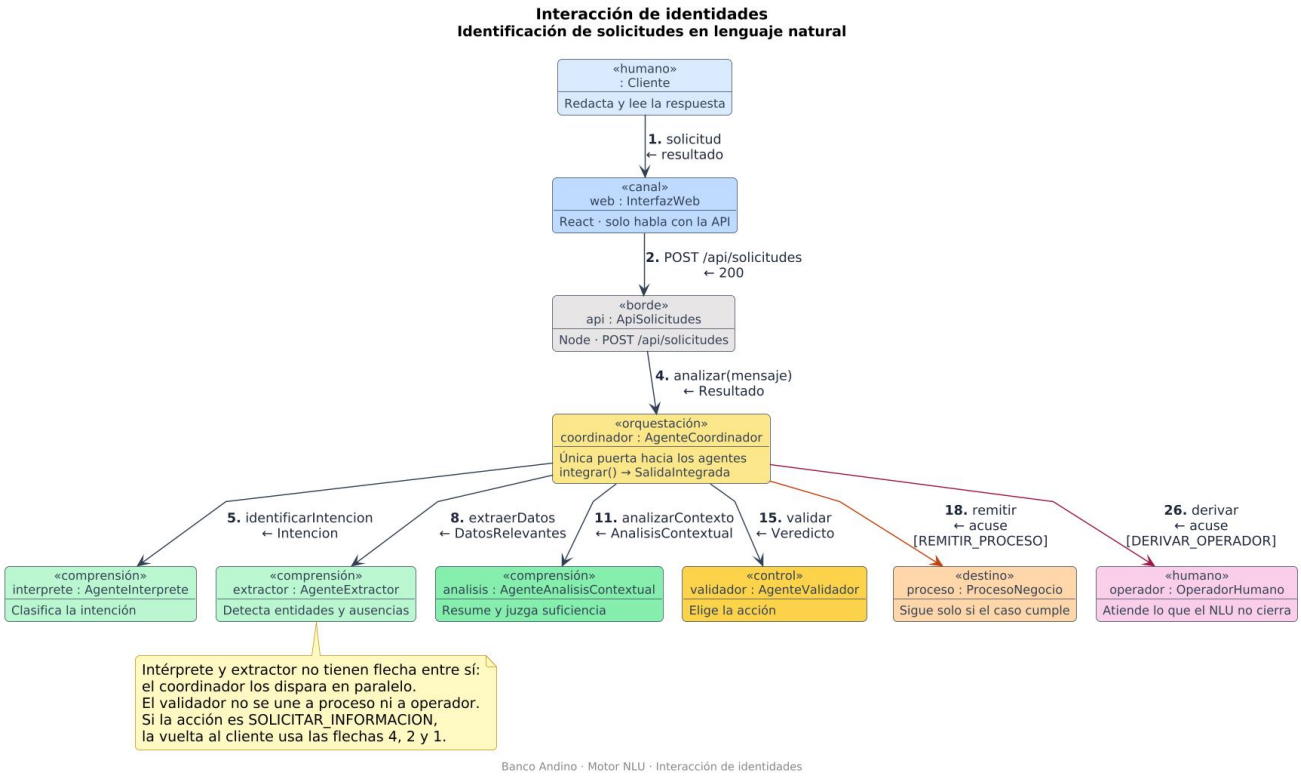


Figura 3. Interacción de identidades del motor NLU. La ausencia de flecha es una regla.

Fuente PlantUML — interacción de identidades

```
' Interacción de identidades — Banco Andino
' Cada caja es una identidad (instancia : tipo).
' Cada flecha es un vínculo permitido. Si dos identidades no están unidas, no se hablan.
' La numeración coincide con el diagrama de secuencia.
' El detalle de cada mensaje está en diagrama_interaccion_identidades.md.

@startuml interaccion_identidades
title Interacción de identidades\n<size:13>Identificación de solicitudes en lenguaje natural</size>

skinparam shadowing false
skinparam roundcorner 8
skinparam linetype polyline
```

```
skinparam nodesep 18
skinparam ranksep 28
skinparam dpi 140

skinparam object {
  BorderColor #334155
  FontColor #0F172A
  FontSize 13
  AttributeFontSize 11
  AttributeFontColor #334155
}
skinparam package {
  BorderColor #64748B
  FontColor #0F172A
  FontStyle bold
}
skinparam arrow {
  Color #334155
  FontColor #1E293B
  FontSize 12
  MessageAlignment left
}
skinparam note {
  BackgroundColor #FEF9C3
  BorderColor #CA8A04
}

together {
  object "«humano»\n: Cliente" as cliente #DBEAFE {
    Redacta y lee la respuesta
  }
  object "«canal»\nweb : InterfazWeb" as web #BFDBFE {
    React · solo habla con la API
  }
  object "«borde»\napi : ApiSolicitudes" as api #E7E5E4 {
    Node · POST /api/solicitudes
  }
  object "«orquestación»\ncoordinador : AgenteCoordinador" as coord #FDE68A {
    Única puerta hacia los agentes
    integrar() → SalidaIntegrada
  }
}

together {
  object "«comprensión»\ninterprete : AgenteInterprete" as interprete #BBF7D0 {
    Clasifica la intención
  }
  object "«comprensión»\nextractor : AgenteExtractor" as extractor #BBF7D0 {
    Detecta entidades y ausencias
  }
  object "«comprensión»\nanalisis : AgenteAnalisisContextual" as analisis #86EFAC {
    Resume y juzga suficiencia
  }
  object "«control»\nvalidador : AgenteValidador" as validador #FCD34D {
    Elige la acción
  }
}

together {
  object "«destino»\nproceso : ProcesoNegocio" as proceso #FED7AA {
    Sigue solo si el caso cumple
  }
  object "«humano»\noperador : OperadorHumano" as operador #FBCFE8 {
    Atiende lo que el NLU no cierra
  }
}

cliente --> web : **1.** solicitud\n← resultado
web --> api : **2.** POST /api/solicitudes\n← 200
api --> coord : **4.** analizar(mensaje)\n← Resultado

coord --> interprete : **5.** identificarIntencion\n← Intencion
coord --> extractor : **8.** extraerDatos\n← DatosRelevantes
coord --> analisis : **11.** analizarContexto\n← AnalisisContextual
coord --> validador : **15.** validar\n← Veredicto
```

```
coord -[#C2410C]-> proceso : **18.** remitir\n← acuse\n[REMITIR_PROCESO]
coord -[#9D174D]-> operador : **26.** derivar\n← acuse\n[DERIVAR_OPERADOR]

note bottom of extractor
  Intérprete y extractor no tienen flecha entre sí:
  el coordinador los dispara en paralelo.
  El validador no se une a proceso ni a operador.
  Si la acción es SOLICITAR_INFORMACION,
  la vuelta al cliente usa las flechas 4, 2 y 1.
end note

footer Banco Andino · Motor NLU · Interacción de identidades

@enduml
```

9. Secuencia de la identificación

El flujo empieza cuando el cliente escribe en lenguaje natural y termina en una de tres salidas. La interfaz no llama a los agentes. El paralelo existe solo entre intérprete y extractor. El análisis contextual es posterior porque necesita las dos salidas para resumir y para marcar huecos o contradicciones.

1. Recepción. El cliente envía el mensaje. La API lo deja en estado RECIBIDA y llama a analizar(mensaje).
2. Intención y datos, en paralelo. El intérprete devuelve Intencion. El extractor devuelve DatosRelevantes.
3. Suficiencia y ambigüedad. El análisis contextual recibe el mensaje, la intención y los datos, y devuelve AnalisisContextual.
4. Integración. El coordinador arma una sola SalidaIntegrada.
5. Validación y destino. El validador devuelve un Veredicto. El coordinador ejecuta la acción y la API responde 200 con el estado correspondiente.

Acción	Condición en el ejemplo	Respuesta al cliente
REMITIR_PROCESO	La salida cumple los requisitos del caso.	200 con estado REMITIDA, resumen e intención. El proceso acusa recibo.
SOLICITAR_INFORMACION	Faltan datos que el cliente puede aclarar.	200 con estado INCOMPLETA, resumen y preguntas. En el ejemplo: caso, monto y fecha.
DERIVAR_OPERADOR	La ambigüedad no se resuelve con otra pregunta, o la confianza no alcanza.	200 con estado DERIVADA, resumen y motivo. El operador acusa la toma.

La secuencia fija el recorrido de una solicitud ya recibida. La persistencia, la autenticación y el alojamiento del modelo están en la arquitectura del capítulo 10, sin cambiar el orden de los agentes ni las tres salidas.

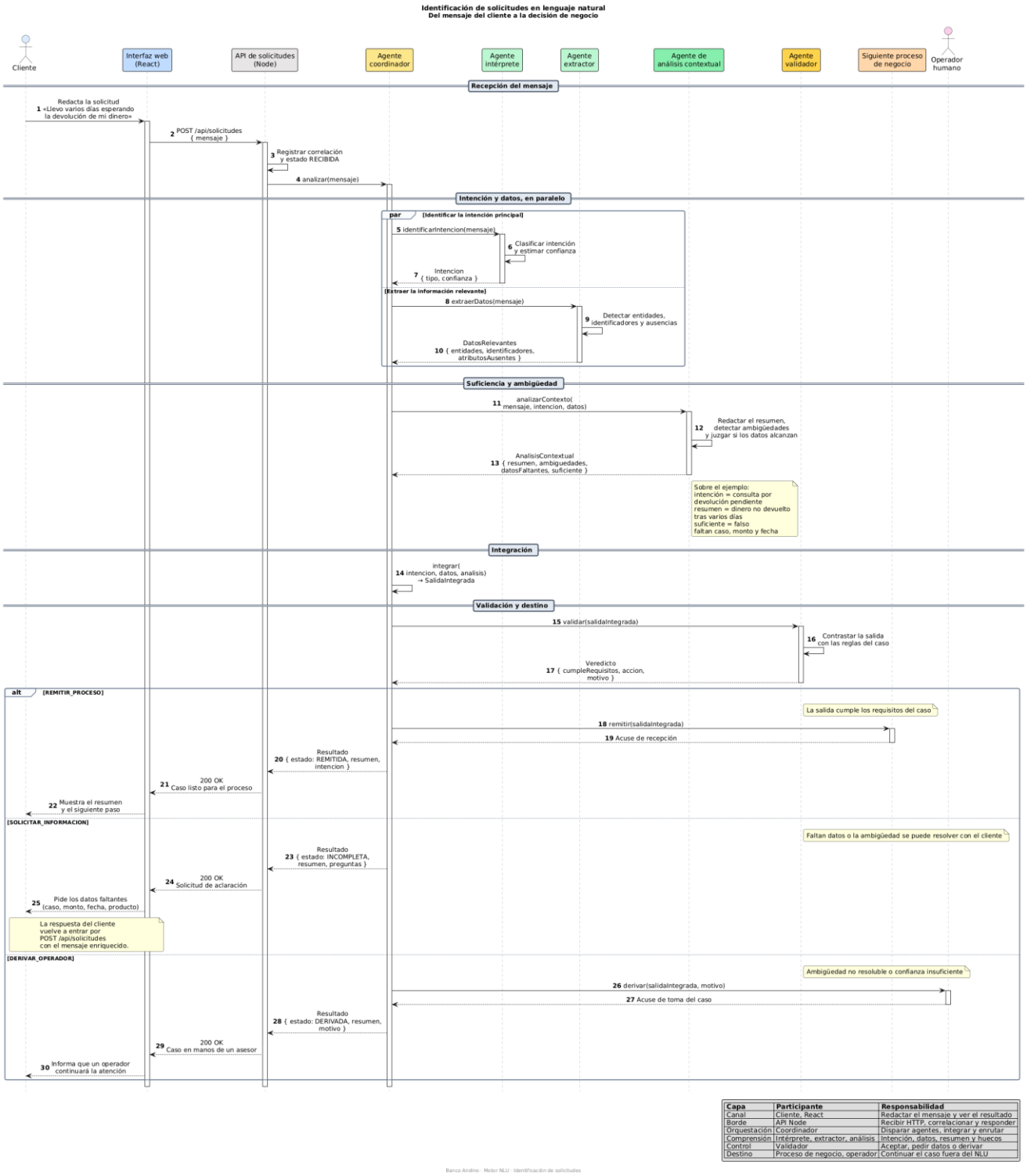


Figura 4. Secuencia de la identificación de solicitudes, del mensaje del cliente a la decisión.

Fuente PlantUML — diagrama de secuencia

```
' Identificación de solicitudes en lenguaje natural — Banco Andino
' Capas previstas: React (interfaz) y Node (API + agentes).
' La interpretación y la extracción corren en paralelo sobre el mensaje crudo.
' El análisis contextual las reconcilia. El validador decide el destino.
'
' Contratos que el diagrama fija para la implementación:
'   Intencion          { tipo, confianza }
'   DatosRelevantes    { entidades, identificadores, atributosAusentes }
'   AnalisisContextual { resumen, ambiguedades, datosFaltantes, suficiente }
'   SalidaIntegrada    { intencion, datos, analisis }
'   Veredicto          { cumpleRequisitos, accion, motivo }
'     accion: REMITIR_PROCESO | SOLICITAR_INFORMACION | DERIVAR_OPERADOR

@startuml secuencia_identificacion_solicitudes
```

```
title Identificación de solicitudes en lenguaje natural\n<size:13>Del mensaje del cliente a la decisión de
negocio</size>

skinparam shadowing false
skinparam roundcorner 6
skinparam maxMessageSize 220
skinparam sequenceMessageAlign center
hide footbox

skinparam sequence {
  ArrowThickness 1
  ParticipantBorderThickness 1
  LifeLineBorderColor #94A3B8
  LifeLineBackgroundColor #FFFFFF
  GroupBorderColor #64748B
  GroupBackgroundColor #F8FAFC
  DividerBorderColor #334155
  DividerBackgroundColor #E2E8F0
}

actor "Cliente" as Cliente #DBEAFE
participant "Interfaz web\n(React)" as Web #BFDBFE
participant "API de solicitudes\n(Node)" as API #E7E5E4
participant "Agente\ncoordinador" as Coord #FDE68A
participant "Agente\nintérprete" as Interp #BBF7D0
participant "Agente\nextractor" as Ext #BBF7D0
participant "Agente de\nanálisis contextual" as Ctx #86EFAC
participant "Agente\nvalidador" as Val #FCD34D
participant "Siguiente proceso\nde negocio" as Biz #FED7AA
actor "Operador\nhumano" as Op #FBCFE8

autonumber

== Recepción del mensaje ==

Cliente -> Web: Redacta la solicitud\n«Llevo varios días esperando\nla devolución de mi dinero»
activate Web

Web -> API: POST /api/solicitudes\n{ mensaje }
activate API

API -> API: Registrar correlación\ny estado RECIBIDA
API -> Coord: analizar(mensaje)
activate Coord

== Intención y datos, en paralelo ==

par Identificar la intención principal
  Coord -> Interp: identificarIntencion(mensaje)
  activate Interp
  Interp -> Interp: Clasificar intención\ny estimar confianza
  Interp --> Coord: Intencion\n{ tipo, confianza }
  deactivate Interp
else Extraer la información relevante
  Coord -> Ext: extraerDatos(mensaje)
  activate Ext
  Ext -> Ext: Detectar entidades,\nidentificadores y ausencias
  Ext --> Coord: DatosRelevantes\n{ entidades, identificadores,\natributosAusentes }
  deactivate Ext
end

== Suficiencia y ambigüedad ==

Coord -> Ctx: analizarContexto(\nmensaje, intencion, datos)
activate Ctx
Ctx -> Ctx: Redactar el resumen,\ndetectar ambigüedades\ny juzgar si los datos alcanzan
Ctx --> Coord: AnalisisContextual\n{ resumen, ambiguedades,\ndatosFaltantes, suficiente }
deactivate Ctx

note right of Ctx
  Sobre el ejemplo:
  intención = consulta por
  devolución pendiente
```

```
  resumen = dinero no devuelto
  tras varios días
  suficiente = falso
  faltan caso, monto y fecha
end note

== Integración ==

Coord -> Coord: integrar(\nintencion, datos, analisis)\n→ SalidaIntegrada

== Validación y destino ==

Coord -> Val: validar(salidaIntegrada)
activate Val
Val -> Val: Contrastar la salida\ncon las reglas del caso
Val --> Coord: Veredicto\n{ cumpleRequisitos, accion, motivo }
deactivate Val

alt REMITIR_PROCESO
  note right of Val: La salida cumple los requisitos del caso
  Coord -> Biz: remitir(salidaIntegrada)
  activate Biz
  Biz --> Coord: Acuse de recepción
  deactivate Biz
  Coord --> API: Resultado\n{ estado: REMITIDA, resumen, intencion }
  API --> Web: 200 OK\nCaso listo para el proceso
  Web --> Cliente: Muestra el resumen\ny el siguiente paso

else SOLICITAR_INFORMACION
  note right of Val: Faltan datos o la ambigüedad se puede resolver con el cliente
  Coord --> API: Resultado\n{ estado: INCOMPLETA,\nresumen, preguntas }
  API --> Web: 200 OK\nSolicitud de aclaración
  Web --> Cliente: Pide los datos faltantes\n(caso, monto, fecha, producto)

  note over Cliente, Web
    La respuesta del cliente
    vuelve a entrar por
    POST /api/solicitudes
    con el mensaje enriquecido.
  end note

else DERIVAR_OPERADOR
  note right of Val: Ambigüedad no resoluble o confianza insuficiente
  Coord -> Op: derivar(salidaIntegrada, motivo)
  activate Op
  Op --> Coord: Acuse de toma del caso
  deactivate Op
  Coord --> API: Resultado\n{ estado: DERIVADA, resumen, motivo }
  API --> Web: 200 OK\nCaso en manos de un asesor
  Web --> Cliente: Informa que un operador\ncontinuará la atención
end

deactivate Coord
deactivate API
deactivate Web

legend right
  |= Capa |= Participante |= Responsabilidad |
  | Canal | Cliente, React | Redactar el mensaje y ver el resultado |
  | Borde | API Node | Recibir HTTP, correlacionar y responder |
  | Orquestación | Coordinador | Disparar agentes, integrar y enrutar |
  | Comprensión | Intérprete, extractor, análisis | Intención, datos, resumen y huecos |
  | Control | Validador | Aceptar, pedir datos o derivar |
  | Destino | Proceso de negocio, operador | Continuar el caso fuera del NLU |
endlegend

footer Banco Andino · Motor NLU · Identificación de solicitudes

@enduml
```

10. Arquitectura de despliegue

Hay dos vistas de despliegue y un invariante. El invariante es el de los capítulos 8 y 9: una sola puerta de entrada, el coordinador como único acceso a los agentes, y tres salidas distintas. La primera vista es el despliegue tentativo en AWS, con la API como único ingreso a la VPC. La segunda separa cada capacidad en un microservicio con su propia base, para cuando haya que escalar el modelo o aislar al validador. El primer código puede vivir en menos contenedores —React y un proceso Node— siempre que las llamadas respeten ese grafo.

10.1 Despliegue tentativo en AWS

El cliente entra por Route 53, WAF y CloudFront. CloudFront entrega el React desde S3 y reenvía /api/* a API Gateway. Cognito autoriza al cliente antes de que la petición cruce a la VPC. API Gateway llega solo a la API Node, por VPC Link. Esa API es la única tarea con entrada desde el borde.

La API guarda el estado de la solicitud en Aurora y llama a analizar(mensaje) en el coordinador. El coordinador dispara intérprete y extractor en paralelo, después el análisis contextual y al final el validador. Intérprete, extractor y análisis llaman a Bedrock. El validador no llama al modelo: contrasta la salida con las reglas del caso. Aurora, Cognito, WAF y CloudWatch cubren estado, acceso, borde y trazas. No son identidades del flujo.

Identidad	Dónde queda en AWS
Cliente	Persona frente a CloudFront.
Interfaz web	React en S3, servido por CloudFront.
API de solicitudes	API Gateway más la tarea Fargate de la API Node.
Coordinador	Tarea Fargate. Solo la API la invoca.
Intérprete, extractor y análisis	Tareas Fargate del grupo Comprensión. Solo ellas usan Bedrock.
Validador	Tarea Fargate del grupo Control. No llama al modelo ni publica eventos.
Proceso de negocio	Consumidor del evento en EventBridge.
Operador	Persona que toma el mensaje dejado en SQS.

Acción	Camino en AWS	Qué ve el cliente
REMITIR_PROCESO	El coordinador publica en EventBridge y espera el acuse del siguiente proceso.	200 con estado REMITIDA.
SOLICITAR_INFORMACION	No hay otro servicio. El resultado vuelve por la misma llamada analizar.	200 con estado INCOMPLETA y las preguntas.
DERIVAR_OPERADOR	El coordinador deja el caso en SQS. El acuse es el de haberlo encolado.	200 con estado DERIVADA.

- Una sola entrada a la VPC. El VPC Link termina en la API. Coordinador, agentes, validador y Aurora no se publican.
- Bedrock no lo llama la API, ni el coordinador, ni el validador.
- El paralelo es de red: intérprete y extractor son dos tareas porque el coordinador las llama a la vez. El análisis espera a que ambas respondan.
- El estado lo escribe la API. Aurora guarda la correlación y el paso de RECIBIDA a REMITIDA, INCOMPLETA o DERIVADA. Los agentes devuelven su objeto y no abren la base.
- Las tareas consumen Bedrock, SQS y EventBridge por endpoints de VPC. Esos endpoints no se dibujan para no tapar el recorrido de la solicitud.

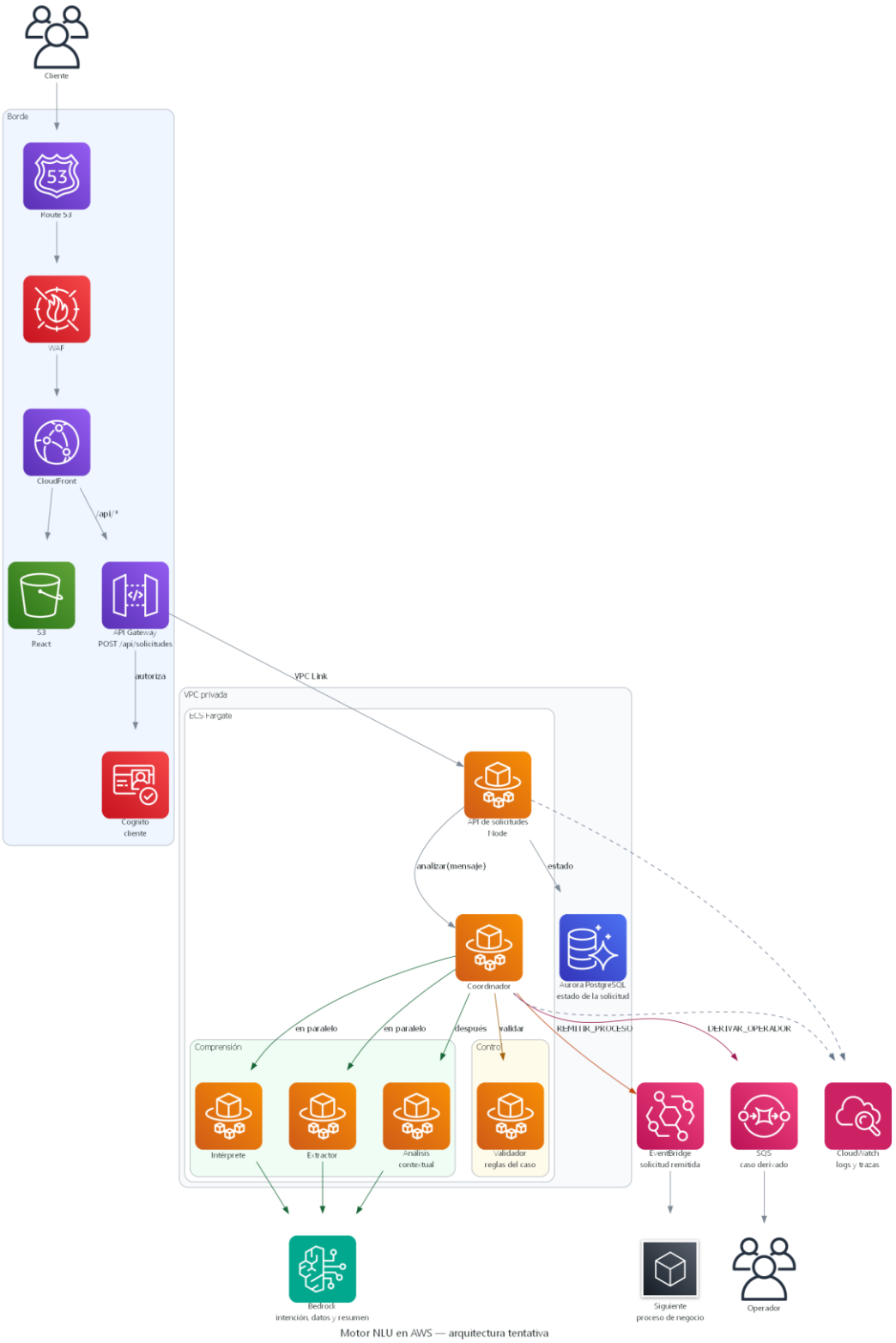


Figura 5. Arquitectura AWS tentativa del motor NLU. Una sola entrada a la VPC y tres salidas.

La gráfica se genera con el script arquitectura_aws.py de la librería Diagrams, desde la carpeta del proyecto NLU y con Graphviz instalado.

10.2 Microservicios, cada uno con su base

Cuando cada capacidad se separa, el frontend sigue hablando solo con la API, cada microservicio tiene su propia base y el borde usa los mismos servicios de AWS. En esta vista, API Gateway llega a Requests API a

través de un Application Load Balancer, que reparte únicamente hacia esa API. Cognito autoriza la llamada y no es un salto hacia el balanceador. El coordinador sigue siendo el único que llama a los demás servicios.

Microservicio	Base propia	Qué guarda
Requests API	Aurora PostgreSQL	Request
Coordinator	DynamoDB	Interpretation ya integrada
Interpreter	DynamoDB	intentType y confidence
Extractor	DynamoDB	entities e identifiers
Context analysis	DynamoDB	summary, ambiguities, missingFields y sufficient
Validator	DynamoDB	Decision

Intérprete, extractor y análisis contextual llaman a Bedrock y no se llaman entre sí. El validador no usa el modelo. REQUEST_INFORMATION vuelve por la misma llamada. FORWARD_TO_PROCESS sale por EventBridge hacia el proceso de negocio. ESCALATE_TO_OPERATOR deja el caso en SQS para el operador. La API proyecta esa acción en Request.status.

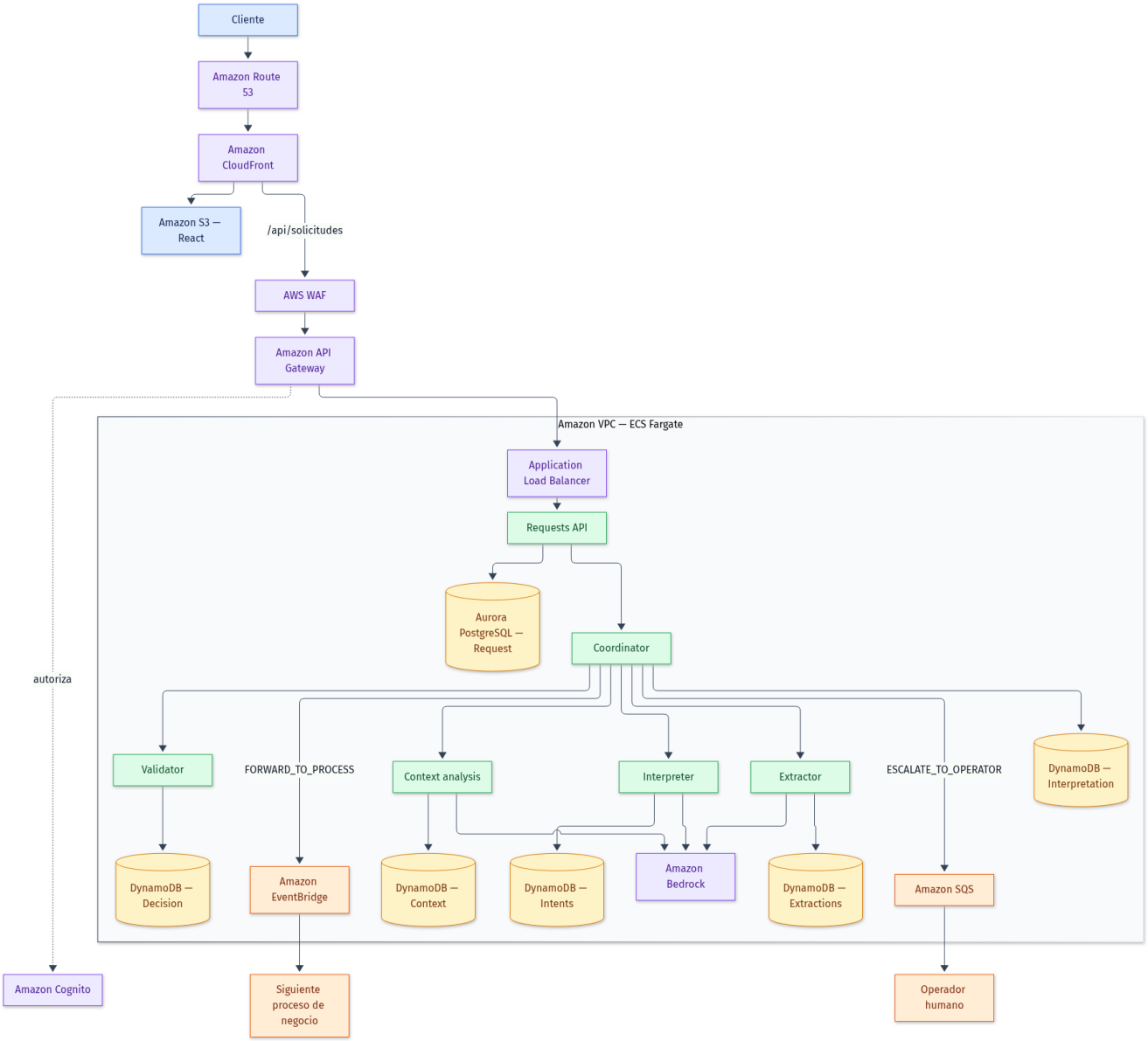


Figura 6. Microservicios del motor NLU en AWS. Cada servicio tiene su base; el cliente entra por CloudFront y la API es la única tarea publicada.

Fuente Mermaid — arquitectura de microservicios

```
%%{init: {"theme": "base", "themeVariables": {"background": "#ffffff", "primaryColor": "#F8FAFC",
"primaryBorderColor": "#334155", "primaryTextColor": "#0F172A", "lineColor": "#334155", "clusterBkg": "#F8FAFC",
"clusterBorder": "#64748B", "fontSize": "16px"}}}%%
flowchart TB
    Cliente["Cliente"] --> R53["Amazon Route 53"]
    R53 --> CF["Amazon CloudFront"]
    CF --> S3["Amazon S3 — React"]
    CF -->|"/api/solicitudes"| WAF["AWS WAF"]
    WAF --> APIGW["Amazon API Gateway"]
    APIGW -.->|"autoriza"| COG["Amazon Cognito"]
    APIGW --> ALB["Application Load Balancer"]

    subgraph VPC["Amazon VPC — ECS Fargate"]
        ALB --> API["Requests API"]
        API --> DB_REQ[("Aurora PostgreSQL — Request")]
        API --> COORD["Coordinator"]
        COORD --> DB_READ[("DynamoDB — Interpretation")]
        COORD --> INTERP["Interpreter"]
        COORD --> EXTR["Extractor"]
        COORD --> CTX["Context analysis"]
        COORD --> VAL["Validator"]
        INTERP --> DB_INT[("DynamoDB — Intents")]
        EXTR --> DB_EXT[("DynamoDB — Extractions")]
        CTX --> DB_CTX[("DynamoDB — Context")]
        VAL --> DB_DEC[("DynamoDB — Decision")]
        INTERP --> BED["Amazon Bedrock"]
        EXTR --> BED
        CTX --> BED
        COORD -->|"FORWARD_TO_PROCESS"| EB["Amazon EventBridge"]
        COORD -->|"ESCALATE_TO_OPERATOR"| SQS["Amazon SQS"]
    end

    EB --> PROC["Siguiente proceso de negocio"]
    SQS --> OP["Operador humano"]

    classDef canal fill:#dbeafe,stroke:#2563eb,color:#172554
    classDef borde fill:#ede9fe,stroke:#7c3aed,color:#4c1d95
    classDef servicio fill:#dcfce7,stroke:#16a34a,color:#14532d
    classDef datos fill:#fef3c7,stroke:#d97706,color:#78350f
    classDef salida fill:#ffedd5,stroke:#ea580c,color:#7c2d12

    class Cliente,S3 canal
    class R53,CF,WAF,APIGW,COG,ALB,BED borde
    class API,COORD,INTERP,EXTR,CTX,VAL servicio
    class DB_REQ,DB_READ,DB_INT,DB_EXT,DB_CTX,DB_DEC datos
    class EB,SQS,PROC,OP salida
```

Las dos vistas conviven así: el despliegue tentativo escribe el estado de la solicitud en Aurora y mantiene a los agentes detrás del coordinador. La vista de microservicios añade una base por servicio y sitúa el balanceador entre API Gateway y Requests API. En las dos, solo la API se publica, solo el coordinador llama a los agentes, solo la comprensión usa Bedrock y las tres acciones conservan su camino.

11. Correspondencia con los artefactos del taller

Cada capítulo de esta especificación sale de un artefacto ya construido. La tabla permite volver del requisito a la fuente.

Capítulo	Artefacto
3. Funcionalidades, beneficios y recorrido	taller_hito_1/actividad_2/respuestas_actividad_2.md, prompts 1 a 3
4. Casos de uso	respuestas_actividad_2.md, prompts 4 y 5; casos_de_uso.puml y casos_de_uso.png
7. Modelo de datos	respuestas_actividad_2.md, prompt 6, diagrama Mermaid
8. Interacción de identidades	diagrama_interaccion_identidades.md, interaccion_identidades.puml y su PNG

Capítulo	Artefacto
9. Secuencia	diagrama_secuencia.md, secuencia_identificacion_solicitudes.puml y su PNG
10.1 Despliegue tentativo	arquitectura_aws.md, arquitectura_aws.py y arquitectura_nlu_aws.png
10.2 Microservicios	respuestas_actividad_2.md, prompt 7, diagrama Mermaid

Los diagramas de casos de uso, de interacción y de secuencia quedan en el documento como gráfica y como fuente PlantUML. El modelo de datos y la arquitectura de microservicios quedan como gráfica y como fuente Mermaid. La arquitectura AWS tentativa queda como la gráfica generada por Diagrams.

Versión 1.0. Consolida el trabajo del motor NLU hasta el 12 de noviembre de 2026.